

Stream API — 컬렉션 데이터 처리 파이프라인

컬렉션을 선언적으로 가공하는 파이프라인. filter → map → collect 체이닝이 핵심.

계층 위치: `collection.stream()` → 중간 연산(intermediate) → 최종 연산(terminal)

Stream의 3가지 특성

특성	설명
원본 불변	원본 컬렉션을 수정하지 않고 새로운 결과를 생성
일회용	최종 연산 후 재사용 불가 (다시 <code>stream()</code> 호출 필요)
지연 평가	중간 연산은 최종 연산이 호출될 때까지 실행되지 않음

핵심 메서드

중간 연산 (Intermediate) — Stream을 반환, 체이닝 가능

메서드	설명	시간복잡도
<code>filter(Predicate)</code>	조건에 맞는 요소만 통과	$O(n)$
<code>map(Function)</code>	각 요소를 변환	$O(n)$
<code>sorted()</code>	자연순 정렬	$O(n \log n)$
<code>sorted(Comparator)</code>	커스텀 정렬	$O(n \log n)$
<code>distinct()</code>	중복 제거	$O(n)$
<code>limit(n)</code>	처음 n 개만	$O(n)$
<code>flatMap(Function)</code>	중첩 구조 평탄화	$O(n)$

최종 연산 (Terminal) — 결과를 반환, 파이프라인 종료

메서드	설명	시간복잡도
<code>collect(Collectors.toList())</code>	결과를 List로 수집	$O(n)$
<code>toList()</code>	결과를 List로 수집 (Java 16+, 불변)	$O(n)$
<code>forEach(Consumer)</code>	각 요소에 대해 동작 수행	$O(n)$
<code>count()</code>	요소 개수 반환 (long)	$O(n)$
<code>findFirst()</code>	첫 번째 요소 반환 (Optional)	$O(1)$
<code>reduce(identity, BinaryOperator)</code>	요소를 하나로 합침	$O(n)$

collect(Collectors.toList()) vs toList()

```
// Java 8~15: collect 사용
List<Integer> list1 = stream.collect(Collectors.toList()); // 가변 리스트

// Java 16+: toList() 사용 가능
List<Integer> list2 = stream.toList(); // 불변 리스트 (add/remove 불가)

// 코딩테스트에서는 collect(Collectors.toList())가 안전 (어떤 버전에서든 동작)
```

패턴

코딩테스트 출제 빈도: 패턴 1중복 제거 + 정렬 > 패턴 2조건 필터링 > 패턴3 변환 > 패턴 4누적 계산

패턴 1: 중복 제거 + 정렬 — distinct + sorted + collect

언제 쓰는가

- 중복을 제거하고 정렬된 결과가 필요할 때

실무에서는

- 태그 목록 정리, 중복 제거된 검색 추천어

```
import java.util.*;
import java.util.stream.*;

public class SortedDistinct {
    public static void main(String[] args) {
        List<String> tags = Arrays.asList("java", "python", "java", "c++",
        "python", "go");

        List<String> uniqueSorted = tags.stream()
            .distinct() // 중복 제거: [java, python, c++, go]
            .sorted() // 정렬: [c++, go, java, python]
            .collect(Collectors.toList());

        System.out.println(uniqueSorted); // [c++, go, java, python]
    }
}
```

패턴 2: 조건 필터링 — filter + collect

언제 쓰는가

- 조건에 맞는 요소만 추출할 때

실무에서는

- 활성 사용자 필터링, 주문 상태별 필터링

```
import java.util.*;
import java.util.stream.*;

public class FilterExample {
    public static void main(String[] args) {
        List<Integer> scores = Arrays.asList(45, 78, 92, 33, 88, 61, 55);

        // 60점 이상만 추출
        List<Integer> passed = scores.stream()
            .filter(score -> score >= 60)
            .collect(Collectors.toList());

        System.out.println(passed); // [78, 92, 88, 61]

        // 개수만 필요하면 count()
        long passedCount = scores.stream()
            .filter(score -> score >= 60)
            .count();

        System.out.println("합격자 수: " + passedCount); // 합격자 수: 4
    }
}
```

패턴 3: 변환 — map + collect

언제 쓰는가

- 각 요소를 다른 형태로 변환해서 새 리스트를 만들 때

실무에서는

- Entity → DTO 변환, 응답 객체 매핑

```
import java.util.*;
import java.util.stream.*;

public class MapTransform {
    public static void main(String[] args) {
        List<String> names = Arrays.asList("alice", "bob", "charlie");

        // 첫 글자 대문자로 변환
        List<String> capitalized = names.stream()
            .map(name -> name.substring(0, 1).toUpperCase() +
```

```

name.substring(1))
    .collect(Collectors.toList());

System.out.println(capitalized); // [Alice, Bob, Charlie]

// 문자열 → 길이로 변환
List<Integer> lengths = names.stream()
    .map(String::length)
    .collect(Collectors.toList());

System.out.println(lengths); // [5, 3, 7]
}
}

```

패턴 4: 누적 계산 — reduce

언제 쓰는가

- 모든 요소를 하나의 값으로 합칠 때 (합계, 최대/최소 등)

실무에서는

- 총 매출액 계산, 최솟값/최댓값 산출

```

import java.util.*;

public class ReduceExample {
    public static void main(String[] args) {
        List<Integer> prices = Arrays.asList(1200, 3500, 800, 5000, 2200);

        // 합계
        int total = prices.stream()
            .reduce(0, Integer::sum);
        System.out.println("총합: " + total); // 총합: 12700

        // 최대값
        int max = prices.stream()
            .reduce(Integer.MIN_VALUE, Integer::max);
        System.out.println("최대: " + max); // 최대: 5000

        // 최소값
        int min = prices.stream()
            .reduce(Integer.MAX_VALUE, Integer::min);
        System.out.println("최소: " + min); // 최소: 800

        // 곱셈 (identity = 1)
        List<Integer> nums = Arrays.asList(1, 2, 3, 4, 5);
        int product = nums.stream()
            .reduce(1, (a, b) -> a * b);
        System.out.println("곱: " + product); // 곱: 120
    }
}

```

```
}
}
```

주의사항

Stream은 처리 파이프라인이지, 데이터 저장소가 아니다

```
List<Integer> list = Arrays.asList(1, 2, 3);
var stream = list.stream().filter(n -> n > 1);
// stream 자체는 아직 아무것도 실행하지 않음 (지연 평가)
// 최종 연산을 호출해야 비로소 실행됨
List<Integer> result = stream.collect(Collectors.toList()); // 이때 실행

// 재사용 불가
// stream.count(); // IllegalStateException 발생!
```

for문 vs Stream 성능 — 코딩테스트에서는 차이 없음

- 데이터가 수만 건 이하면 성능 차이는 무시할 수준
- 코딩테스트에서는 **가독성과 작성 속도** 기준으로 선택
- 단순 반복은 for문이 빠를 수 있고, 복잡한 체이닝은 Stream이 깔끔

디버깅 팁: peek()으로 중간 결과 확인

```
import java.util.*;
import java.util.stream.*;

public class PeekDebug {
    public static void main(String[] args) {
        List<Integer> nums = Arrays.asList(1, 2, 3, 4, 5);

        List<Integer> result = nums.stream()
            .filter(n -> n % 2 == 0)
            .peek(n -> System.out.println("필터 통과: " + n)) // 필터 통과:
2, 필터 통과: 4
            .map(n -> n * 10)
            .peek(n -> System.out.println("변환 후: " + n)) // 변환 후:
20, 변환 후: 40
            .collect(Collectors.toList());

        System.out.println(result); // [20, 40]
    }
}
```